THE HONG KONG POLYTECHNIC UNIVERSITY
DEPARTMENT OF COMPUTING

FINAL PROJECT · SEMESTER 1, 2026/27

Final Project

Project details & assessment guide

One complete web application. Any topic, any stack — your design, your code.

10%	1-2	10 min	100
OF THE SUBJECT GRADE	STUDENTS PER GROUP	RECORDED DEMO	MARKS + UP TO 10 BONUS

INSIDE THIS GUIDE

01 The brief
02 What to build
03 Milestones & deliverables
04 How you are assessed
05 Marking criteria in detail
06 Group work, integrity & AI
07 Support & FAQ
A Marking sheet
B Contribution declaration

01 · FORM A GROUP
One or two students; agree roles before Week 8.

02 · PICK A TOPIC
Any code-based web application you care about.

03 · SUBMIT THE OUTLINE
One page, via Canvas, by the end of Week 8.

04 · BUILD & DEMO
Weeks 9–15: build, test, deploy and present.

Prof. Lei Yang · Department of Computing · ray.yang@polyu.edu.hk

Version 1.2 · November 2026

01 The brief

Design and build one complete web application. The topic is yours; the standard is a working product that a real user can operate — not a set of disconnected features.

This is the only project in COMP3421. Working alone or in a pair, you will take an idea from concept to a running web application, then demonstrate and defend it. You will use the stack taught in the course — HTML, CSS and JavaScript on the client, plus a server and a database for pairs (optional for solo) — and the workflow used in lectures and labs. You will design the architecture, build and test the app, and deploy it if claiming the +5 deployment bonus. You may choose **any topic, any platform and any framework or library**. Vibe coding is allowed; hand-coding earns a bonus (page 3). Two-person groups must build both sides and a database; a solo project may omit the server, but server-side marks are then unavailable.

WHY THIS PROJECT

AI can generate code in seconds. What it cannot do for you is decide what to build, structure the system, judge whether the result is correct and secure, or explain the trade-offs. That is what this project assesses.

What you will deliver

CLIENT

Front end

A responsive, multi-view interface with real interaction, form validation and no broken flows.

LOGIC

Back end

Application logic and data access behind an API — validation, errors, state changes and persistence.

DATA

Database

A persistent store with a schema and seed data. Required for two-person groups.

BONUS

Deployment

A public URL on a cloud host or the department server. Up to **+5 bonus marks**.

Two bonus opportunities: real deployment **+5** · hand-coded implementation **+5** (no AI-generated code). Maximum project score: 110.

Choose any topic

The topic is completely open. Pick a domain you know, or a problem you would actually use. The examples below are starting points — not a prescribed list.

DOMAIN	A WORKABLE APPLICATION	TECHNICAL HOOKS
E-commerce	Pet shop, bookstore or marketplace with catalogue, search, cart, checkout and orders.	Relational data, auth, transactions, image upload
Community	Discussion board, chatroom or social feed with profiles, posts, comments and notifications.	Real-time (WebSocket), moderation, roles
Booking	Room, studio or event booking with availability, reservations, cancellation and admin.	Dates, conflict checks, validation, email
Learning	Course platform with lessons, quizzes, progress tracking and a dashboard.	Scoring, charts, auth, data visualisation
Dashboard	Data explorer that pulls from a public API and offers filters, charts and saved views.	Async fetch, caching, visualisation
Campus	Lost-and-found, room finder or event board for PolyU students.	Search, uploads, roles, geolocation

THE ONE HARD RULE

It must be a **code-based web application** — not a static brochure site, and not a site assembled with a CMS or website builder such as WordPress, Wix, Squarespace or Shopify. If a visitor can only read pages, or the site is configured rather than built, the brief is not met.

02 What to build

A complete application, not a collection of screens. The baseline defines “complete”; marks above it come from quality, depth and finish.

CLIENT SIDE

Front end

- 3+ functional views (pairs: 6+).
- Responsive from phone to desktop.
- Semantic HTML, CSS layout (Grid/Flexbox).
- JavaScript interactivity and form validation.
- Labels, alt text, keyboard access; no console errors.

SERVER SIDE

Back end

- Application logic behind an API.
- Validate every input on the server.
- Handle errors and return JSON.
- Auth and authorisation for protected actions (recommended).
- Any server platform is allowed.

DATA

Database

- A real database: SQLite, MySQL, PostgreSQL, MongoDB or a managed service.
- Schema and seed data included.
- Reads and writes persist across restarts.
- No secrets or real personal data.

Minimum expected scope

CRITERION	SOLO PROJECT	TWO-PERSON PROJECT
Functional views	3+	6+
Data entities	2+	4+
Core flows	2+ complete CRUD flows	4+ complete CRUD flows
Accounts & roles	Optional	Recommended
Server-side API	Optional	Required
Database	Optional (client storage allowed)	Required
Responsive layout	Required	Required
Testing evidence	Manual test table	Manual + some automated tests

The baseline is guidance, not a checklist. Markers assess quality, not counts; exceeding the baseline with unfinished features does not help. Two-person groups are expected to deliver a broader full-stack application than solo projects, and markers calibrate to group size.

Coding routes

ROUTE	WHAT IT MEANS	STATUS / BONUS
Vibe coding	AI tools may generate some or all of your code. Declare the tools, the parts they generated and how you verified them. You must understand and be able to explain every line of application code you submit.	—
Hand-coding	You write the application code yourself. AI may explain concepts, debug or review, but must **not generate submitted code**. Declare any AI use.	**+5** if verified
CMS / site builder	WordPress, Wix, Squarespace, Shopify, Weebly or similar. The site is configured, not coded.	Not permitted

Using frameworks, libraries and third-party assets is allowed on both code routes — they are not “AI-generated code”. A ready-made template must be declared; imported template code is not “hand-written” by you and may affect the bonus. Verification is explained on page 7.

WHAT COUNTS AS COMPLETE

A new user can finish the main task end-to-end: the data is saved, invalid input is handled, and the app still works after a refresh. A template with your logo, hard-coded sample data, or buttons that do nothing is not complete.

TECHNOLOGY RULES

Any stack is allowed — any framework, library, database and hosting platform. The course teaches HTML/CSS/JavaScript + Node.js/Express, so that is the best-supported path. The rule is simple: your TA must be able to run your app by following `README.md`, and you must be able to explain and defend the core application logic.

03 Milestones & deliverables

Three checkpoints. The outline gets you feedback, the submission is what is marked, and the demo is where you prove it runs.

WEEK 8 · 5%
Project outline
One-page PDF via Canvas: group, title, abstract, scope, stack, data model sketch and roles.

20 DECEMBER · 75%
Final submission
One zip per member: report, demo_video, signed contribution declaration, ai-use.md, source code, README, schema and seed data.

20 DECEMBER · 20%
Video presentation
10 minutes, recorded: overview and a demo of the app. No live session.

Percentages are shares of the 100 project marks. +5 deployment and +5 hand-coding bonus before the deadline; exact dates are on Canvas.

3.1 Project outline — 5%

Due: end of Week 8, via Canvas. File: `COMP3421_Outline_<GroupID>.pdf`.

- **Members:** names, student IDs and tutorial group; solo or pair.
- **Title + abstract** (150–250 words): problem, target users, what the app does.
- **MVP features** and optional stretch features.
- **Stack:** client; server and database (pairs) or client-side storage (solo); deployment plan (optional, +5).
- **Data entities:** 2–5 main things the app stores and how they relate (solo: 2+; pairs: 3+).
- **Roles** and provisional contribution split; **risks** and what you will cut.

The outline is only 5%, but it is your chance to get early feedback. Submit it even if the idea is still moving. If the scope changes substantially, email the instructor and your TA.

3.2 Final submission — 75%

Due: 20 December 2026 (Sunday), 23:59, via Canvas. **One joint report per group.** Each member submits the package separately on Canvas; the two submissions may overlap in content. Do not include `node_modules/`, `.env` files or generated files. Provide test accounts and seed data. Every group submits `contribution-declaration.pdf`: pairs sign the joint statement; a solo student signs only the Solo Project Statement.

SUBMISSION PACKAGE

```
COMP3421_Project_<GroupID>/
├── report.pdf
├── contribution-declaration.pdf
├── ai-use.md
├── README.md
├── source/
│   ├── client/
│   ├── server/
│   ├── database/   (schema.sql, seed.sql)
│   ├── package.json
│   └── .env.example
└── demo_video.mp4  (10 minutes, compulsory)
```

GroupID = each member's name + student ID, e.g. `Amy_230xxxxx_Bob_240xxxx`. README must cover: prerequisites, install, environment variables, database setup, run, test accounts, deployment URL and known limitations.

REPORT MUST INCLUDE

- Cover; abstract and introduction; requirements and feature list.
- Architecture: diagram, stack, components and their responsibilities.
- Data model and API: ERD plus an endpoint table (method, input, output, status).
- Implementation: key flows, non-trivial decisions and code highlights.
- UI/UX: screenshots and evidence of responsive behaviour.
- Testing: plan, cases, results and known bugs.
- Deployment and operations; limitations and future work.
- Individual contributions; references; appendix (user manual, AI use, third-party assets).

Suggested length: 10–15 pages (pair) or 7–10 (solo), excluding appendices. Marks are for content, not length.

3.3 Video presentation — 20%

- **10-minute recorded video** (`demo_video.mp4`), submitted with the package; no live session.
- **Introduce the project**, then show the actual app: main pages, features, functionality.
- **Record the submitted build** from the package — not a private "demo version"; both members appear.
- **Prepare seeded data and test accounts.** Show persistence: reload and re-open the app.
- **Have a backup:** keep a second copy of the file; record locally, not from the cloud.
- **No live coding.** A short code walkthrough may form part of the video; show, don't tell; be ready to explain your code and data.

04 How you are assessed

Your project is marked out of 100 using the marking sheet in Appendix A, then weighted to 10% of your subject grade. Creative work involves judgement; the criteria and band descriptors make that judgement consistent and transparent.

COMPONENT	MARKS	WHAT IT REWARDS
Project outline	5	A clear, feasible plan, submitted on time, with members and roles defined.
Functionality & implementation	50	A working application: logic, features, architecture, client, server/data, robustness.
Project report	25	Design documentation, data model, testing evidence, user manual and signed contribution declaration.
Demonstration	20	A clear 10-minute recorded demo without runtime errors, and both members explaining their part.
Real deployment	+5	A publicly reachable, stable deployment of the submitted build, documented in the README.
Hand-coded implementation	+5	Source code written by the team, with no AI-generated code; verified by declaration and code walkthrough.

The project is worth 10% of COMP3421. Its base is 100 marks, scaled to that 10% weight; bonus marks (max +10) are added to the project score, which can reach 110. The other 90% is the final exam (70%), mid-term (10%) and assignments (10%).

How markers judge quality

Four dimensions run through every component. They are not separate scores; they are the lens markers use when reading the rubric on the next page.

Complexity & scope

How ambitious and difficult the problem is, and how much of it you completed. Judged by interacting parts, not page count.

Aesthetics & usability

Visual hierarchy, consistency, responsiveness, accessibility, feedback and empty/error states. A calm, clear interface beats decoration.

Maturity

Does it work end-to-end under realistic use? Validation, error handling, persistence, reproducible setup, security basics.

Technique

Appropriate use of course concepts, clean architecture, code quality, and evidence of testing and deployment.

Band descriptors

DIMENSION	EXEMPLARY	STRONG	COMPETENT	DEVELOPING
Complexity & scope	Ambitious but completed; several interacting features; data model and user flows beyond the baseline.	Multi-feature app with real integration; baseline met.	Basic CRUD app; limited integration; some features unfinished.	Trivial or largely static; copied rather than built.
Aesthetics & usability	Distinctive, consistent, responsive and accessible; clear feedback and states; easy for a new user.	Clean and consistent; minor rough edges; works on mobile.	Functional UI; inconsistent styling or a weak mobile layout.	Cluttered or broken layout; hard to use.
Maturity	Reliable under edge cases; validation, errors, persistence, tests and a reproducible install.	Works in normal use; minor bugs; some validation.	Main flows work; errors and edge cases unhandled.	Frequent runtime errors; hard to run; incomplete.
Technique	Sophisticated, appropriate use of course concepts; clean architecture; secure, tested and deployed.	Good use of concepts; clear structure; some advanced techniques.	Core concepts used; simple structure; limited depth.	Superficial use; copy-paste; poor code quality.

Band descriptors are a shared language for markers, not a formula. A smaller, polished and completed app can outscore a larger unfinished one.

The marking sheet is completed at the demo and reviewed by the instructor. Individual adjustments are recorded on the sheet; feedback is returned through Canvas.

05 Marking criteria in detail

Indicative sub-marks within each component. Markers use them together with the band descriptors on the previous page — they guide emphasis, not a mechanical checklist.

Functionality & implementation — 50

CRITERION	MARKS	WHAT MARKERS LOOK FOR
Application logic & features	15	Coherent logic; features work end-to-end; scope completed rather than promised.
Architecture & integration	10	Clear client/server separation; sensible components; client and API integrate correctly.
Client-side design & UX	10	Multiple views; responsive; usable navigation; clear feedback; accessible controls.
Server-side design & data	10	API design; validation; error handling; auth where needed; schema fits the data.
Robustness, security & code quality	5	No runtime errors; input handled; secrets protected; readable code; tests.

Project report — 25

CRITERION	MARKS	WHAT MARKERS LOOK FOR
Design & architecture	7	Architecture and component diagrams; choices justified; structure explained.
Data model & API	5	ERD, tables and relationships; endpoint table with methods and inputs.
Implementation & testing	6	Key decisions; test plan, cases and results; known bugs; lessons.
User manual & reproducibility	4	Install, configure, seed the database, test accounts and main flows.
Clarity, professionalism & contribution	3	Structure, writing, figures, citations; contribution declaration consistent.

Demonstration — 20

CRITERION	MARKS	WHAT MARKERS LOOK FOR
Overview	3	Problem, users, architecture and key decisions in 1–2 minutes.
Live or recorded demo	8	No errors; key journeys completed; data persists; all members demo.
Coverage & ease of use	5	All core features shown; understandable without explanation; edge cases handled.
Q&A	4	Accurate answers on code, data and trade-offs; whole-system understanding.

Project outline — 5

CRITERION	MARKS	LOOK FOR
Completeness	2	All required items present, including roles and contribution plan.
Clarity & feasibility	3	Realistic scope; coherent features and data model; convincing plan.

Real deployment — +5 bonus

CRITERION	MARKS	LOOK FOR
Public URL & deployed build	2	Reachable URL serving the submitted build; if there is an API, it is live too — not just a static copy.
Stability & documentation	3	Data survives restarts; stable; no exposed secrets; documented.

Hand-coded implementation — +5 bonus

CRITERION	MARKS	WHAT MARKERS LOOK FOR
Declaration & evidence	2	Signed declaration of no AI-generated code; commit history consistent.
Code walkthrough	3	Each member explains their part and, if asked, edits it; withdrawn if AI code is found.

Sub-marks are indicative and may be adjusted holistically. For example, an ambitious project that is 90% complete may score differently from a simple project that is 100% complete; markers explain the adjustment on the marking sheet.

06 Group work, integrity & AI

A pair can achieve more than a solo project — but only if the work is genuinely shared and each of you can stand behind the whole system.

Working in a pair

- **Agree roles on day one:** front-end lead and back-end lead. Ownership is clear, but both of you integrate, test and review the whole app.
- **Keep one shared Git repository.** Commit often with meaningful messages; the history is evidence of contribution.
- **Write one joint report.** Divide sections, review each other's writing, and state who wrote what.
- **All group members attend the demo** and can run the app from a clean install.
- **If contribution becomes uneven, tell your TA and the instructor early** — not after Week 13.

The report and the contribution declaration

REQUIRED FOR EVERY GROUP

Submit a signed **Contribution Declaration** (Appendix B, `contribution-declaration.pdf`) and summarise it in the report. It records each member's role, components owned, evidence (files/commits), percentage of total work, and the report sections they wrote. Percentages must sum to 100% and match the evidence. Pairs complete the joint statement; solo students complete only the Solo Project Statement. Each member submits the package separately; the same report and code may be submitted by both.

- **One joint report for the pair.** It must contain a short "Individual contributions" section, and the signed declaration must be attached as a separate PDF.
- **The group mark is the starting point.** Individual marks may be adjusted up or down for contribution, demonstrated understanding and the demo. In serious cases, a member may receive a mark that reflects only their verified contribution.
- **Free-riding is not a strategy:** if you cannot explain code you submitted, it affects your individual mark.

Academic integrity

- Your design, code and writing must be created for this project. Declared AI assistance is permitted under the rules below; you may discuss ideas with classmates, but do not share code, reports or assets.
- Do not resubmit a previous course project, a lab solution, or another group's work. Small snippets from lectures and labs may be reused with attribution — not wholesale.
- Cite libraries, frameworks, templates, images, icons and fonts; use licensed or public-domain assets and keep the licence information where required.
- Plagiarism, undeclared AI assistance and fabricated contribution claims may be treated as academic misconduct under University regulations.

AI tools and the hand-coding bonus

VIBE CODING ROUTE

AI tools may generate code. Declare the tool and version, what it was used for, which files or parts it produced, and how you verified it. You remain responsible for every line. You are eligible for all marks except the hand-coding bonus.

HAND-CODING ROUTE

You write the application code yourself. AI may explain concepts, debug or review, but must not generate submitted code. Declare any AI use and claim the bonus in `ai-use.md` and the contribution declaration. Verified by a code walkthrough.

If AI-generated code is found in a project claiming the hand-coding bonus, the bonus is withdrawn. The project itself is still assessed — under the vibe-coding route. A knowingly false hand-coding declaration may also be handled under the University's academic integrity procedures. The final exam rules are unchanged: no electronic devices or AI tools in the exam.

Data, third-party assets & safety

- Use dummy data, never real personal data, passwords or payment details.
- Keep API keys out of the repository; ship `.env.example`, never `.env`.
- No illegal, harmful or harassing content; follow PolyU IT policies and the law.
- Validate and sanitise user input; protect write operations.

07 Support & FAQ

Ask early. A short question in Week 9 is worth more than a crisis in Week 13.

Frequently asked questions

Do we have to use Node.js and Express?

No. Any platform, framework and library is allowed. The course teaches Node.js/Express, so it is the best-supported path, but your TA must be able to run your app from your README.

Can we use React, Vue, Bootstrap, Tailwind or a template?

Yes — declare them and their licences. Frameworks and templates do not earn marks by themselves; what you build, integrate and customise does.

Can we use WordPress, Wix or Shopify?

No. CMS and website-builder platforms are not permitted; the project must be built in code. Code-based frameworks are fine; a static-site generator may be used as a build tool, but the result must still be an interactive app with data and logic.

Can we use Firebase, Supabase or another managed backend?

Yes, as part of your stack. Declare the service and show how it is used. Server-side marks are based on the application logic and data design you build, not on the hosting brand.

Can we use AI / vibe coding?

Yes, with a declaration. If you want the +5 hand-coding bonus, AI must not generate any submitted code — it may only explain, debug or review.

Can our project be a static website or portfolio?

No. It must be an interactive web application with data and logic. A portfolio with accounts and dynamic content could qualify; a set of static pages does not.

Before you submit

- [ ] Group of 1–2 confirmed; roles and contribution plan agreed.
- [ ] Outline submitted on Canvas by the end of Week 8.
- [ ] Topic is an interactive web application, not a CMS site.
- [ ] Coding route declared: vibe coding or hand-coding.
- [ ] Client and server (if used) run from a clean install following README.
- [ ] Database schema and seed data included (if used); test accounts work.
- [ ] Contribution declaration / solo statement signed and attached; report has the contributions section.
- [ ] AI use and third-party assets declared with licences.
- [ ] Responsive check at 360 px and desktop; no console errors.
- [ ] Backup demo video and screenshots ready.
- [ ] Zip opens; no `node_modules` or `.env`; filename correct.
- [ ] Each member of a two-person group submitted the package separately (content may overlap); deadline met.

WHERE TO GET HELP

Instructor: Prof. Lei Yang · `ray.yang@polyu.edu.hk` · PQ 708
Your tutorial TA (see course outline / Canvas)
Canvas: announcements, submission and marks
Course examples: `tagsysx.github.io/comp3421/`

More questions

Can we change the topic after Week 8?

Yes. Email the instructor and TA; a major change may need a short updated outline. Your outline mark is based on what you submitted.

What if our deployment is down during the demo?

Demo a local build from the submitted package. The demo must not depend on the cloud; the deployment bonus is judged on evidence and stability.

How many features or pages do we need?

No fixed number. The baseline on page 3 defines “complete”. A smaller app that is finished, polished and tested scores better than a large one with broken features.

What if my partner does not contribute?

Raise it early with your TA and the instructor. The contribution declaration and demo are used to adjust individual marks; do not wait until the deadline.

Can we submit late?

Follow the subject’s late-submission policy and tell the instructor as soon as possible. Do not assume an extension.

Project health checkpoints

WEEK 9
Lock the scope
Finalise the feature list and data model; create the repository; agree who owns what.

WEEKS 10–11
Build the core
Complete two end-to-end flows; connect the API to the database; commit regularly.

WEEK 12
Harden & rehearse
Validation, responsive checks, tests and README; rehearse the 5–8 minute demo.

THE READINESS TEST

Can a stranger install your app from the README and complete the main task without asking you a question? If yes, you are ready. If not, fix that before you add another feature.

A

Marking sheet

The sheet used by assessors. It is included so you can see exactly how the marks are allocated. One sheet per group; totals are indicative and can be adjusted holistically.

Group No. _____ **Project title** _____

Assessor _____ **Date** _____ **Coding route** ☐ Vibe coding ☐ Hand-coding

Member name	Student ID	Role	Contribution %

Criterion	Marks	Awarded	Comments / evidence
Project outline members, title, abstract, scope, roles	5		
Functionality & implementation application logic · features · architecture · client design · server/data design · robustness	50		
Project report design · data model/API · implementation & testing · user manual · clarity & contribution	25		
Demonstration overview · live or recorded demo · coverage & ease of use · Q&A	20		
Real deployment BONUS public URL · deployed build · stability · documentation	+5		
Hand-coded implementation BONUS declaration & evidence · code walkthrough; no AI-generated code	+5		
TOTAL base /100 + bonus (max +10)	100+10		

Bonus marks are only awarded when the evidence is present. A project without a deployment, or without a verified hand-coding declaration, simply has no bonus in that row; it is not penalised. The two bonuses are independent.

Individual adjustment (if any)	Group mark	Adjustment	Final mark	Reason

Overall comments — strengths, weaknesses and one concrete improvement

Assessor signature _____ Date _____

B Contribution declaration

One declaration per group, signed by every member and submitted as a separate PDF. A two-person group submits one joint report; the declaration is required. Pairs complete the joint statement; a solo student completes only the Solo Project Statement. Each member submits the package separately; the same report and declaration may be included in both submissions.

Group No. _____ **Project title** _____

Tutorial group _____ **Date** _____ **Group size** ☐ 1 ☐ 2

Name	Student ID	Role	Main components / features owned	% of total work

Percentages must sum to 100% and match the evidence. Add rows if needed.

Report sections written by each member

Evidence of contribution (files, commits, issues)

CODING ROUTE AND AI DECLARATION

Route (tick one): ☐ Vibe coding — AI-generated code may be included ☐ Hand-coding — no AI-generated code; +5 bonus

AI tools used: ☐ None ☐ Yes — tools and purpose: _____

If vibe coding: which parts were generated: _____

If hand-coding: AI was used only for (explain / debug / review): _____

JOINT STATEMENT

We confirm that the report and source code submitted for COMP3421 are our own group's work; that the contribution percentages above are accurate and sum to 100%; that all external code, libraries, assets and AI assistance are declared; and that we understand individual marks may differ where contribution differs. We understand that a false declaration may be treated as academic misconduct.

Member	Name	Signature	Date
Member 1			
Member 2 (pairs only)			

SOLO PROJECT STATEMENT

I confirm that I completed this project alone, that the report and source code are my own work, and that all external code, assets and AI assistance are declared.

Name: _____ Signature: _____ Date: _____

FOR INSTRUCTOR / TA USE ONLY — BONUS VERIFICATION

☐ Contribution declaration checked ☐ Code walkthrough completed ☐ Deployment URL verified
☐ No AI-generated code found (hand-coding bonus confirmed) ☐ Contribution percentages sum to 100% Notes: _____

Editable Word version: download the Word template — fill, sign, and save as `contribution-declaration.pdf`.